

Automated Provisioning of Configuration Base States

Ben Frempong, a storage test engineer, continues the Dell story, describing how they automated configuration base states to make the setup for testing much easier.

In mid-2012, our Storage Test team was faced with budget and staffing challenges. We had to devise a sustainable solution to a reality that faces most test organizations at one time or another: how to do more with fewer resources while continuously improving the quality of products we deliver to our customers. The solution required us to reinvent and implement sustainable and innovative automation strategies in our testing process to achieve three key goals:

1. Maximize the utilization of our hardware resources.
2. Improve the efficiency of our test execution staff.
3. Capture automation metrics.

The first task was to establish some standards for our automation initiatives. We chose Python as our standard scripting language and selected a centralized, internally developed automation framework for remote script deployment. Next, we focused on two teams that had similar needs: one local and the other remote. After analyzing their daily tasks, we organized their testing activities into the following workflow sequence:

1. Configure base hardware.
2. Install OS.
3. Install updates (firmware/BIOS/OS device drivers).
4. Install solution/product peripherals.
5. Install and configure solution/product-specific packages.
6. Run a suite of manual and automated test cases.
7. Perform exploratory testing.
8. Tear down and reprovision the system for another configuration.

I observed that the first three steps, the base state of provisioning workflows, was often a manual, time-consuming, and repetitive sequence of tasks with the same operating system, on the same hardware or platform families. There were sometimes minor configuration changes such as add-in storage peripherals, but these changes did not impact the base state of any configuration. Additionally, the base state

configurations often had to be redeployed to validate software build releases or bug fixes during a program's test cycles. An automated process could be used to create disk images, files containing the contents and structure of a disk volume or data storage device, with the necessary base configurations. These images could be used to automatically reprovision each configuration's base state.

Existing imaging solutions at that time worked very well in large-deployment environments but not so well in test environments, where one is continuously reprovisioning the same hardware. Also, they took up to 90 minutes or more, depending on the operating system, to capture an image. It might take several hours to restore non-Windows images such as Linux OS. This explained why testers preferred the manual method. We tried to automate existing off-the-shelf imaging solutions, but no single solution supported automated image capture and restores for all three required system environments that we needed to support: Windows, Linux, and ESXi5. Considering the matrix of hardware that needed to be validated, we would have to either purchase more hardware and assign more configurations per tester, or add more testers. Due to our budget challenges, neither solution was an option.

Instead, we prioritized automating the provisioning workflows with some innovative ideas by developing our own fully automated imaging/redeployment solution. First, we repackaged the kernel for the open-source imaging tool Clonezilla to work with our automation framework. Next, we wrote fully automated imaging and redeployment scripts for Windows, Linux, and ESXi5 images. These scripts automatically capture and redeploy base-state images in 10 to 15 minutes or less.

The results have been positive. One of the project teams had typically required up to three testers per test iteration to cover 12 to 18 configurations. After implementing the automated imaging and restore tasks, the team lead was able to single-handedly execute the test iteration himself without additional resources. He even had more time for exploratory testing because the test execution framework allowed him to launch a sequence of automated tasks, which could run overnight or during lunch.

With our new approach to workflow test automation, we are now able to do more with fewer resources, while giving our skilled staff more time for exploratory testing. This allows us to continue to deliver quality products. We are also able to reduce expenditure and maximize the utilization of our hardware resources. Last, from a process perspective, we are able to use the information about our automation capabilities in our test planning, test strategy, and business decisions.



Testers, programmers, and operations experts can collaborate on the CI process to make build jobs more robust, test results more reliable, and deploys to various environments timely and sane. If the whole team understands their operating environments, they may save many hours spent investigating test failures. For example, DevOps may be able to help create checks for connectivity or configuration before deploying, which eliminates wasted time rejecting builds. Alternatively, testers and programmers can analyze test suites to identify duplicate tests, tests that don't add value, and ways to improve reliability. They can also look for gaps in regression test automation and work to fill them.

DevOps practices help our teams efficiently build regression test code and deploy it as needed to environments that support effective exploratory testing. Skilled DevOps practitioners help us implement tools to support testing all necessary quality attributes such as security, performance, and reliability. These abilities are essential for cross-functional teams to deliver high-quality software that provides business value frequently, at a sustainable pace.

SUMMARY

DevOps is a perfect example of the whole-team approach to quality at work. It brings together generalizing specialists with different T-shaped skills to help ensure several aspects of quality. Even when development and operations are separate departments, they can work together to achieve shared goals. Here are some points about DevOps that we covered in this chapter:

- DevOps is a blend of development, testing, and operations engaged in practices that streamline the delivery process, provide timely feedback to improve cycle time and software quality, and facilitate collaboration among all roles on the team.
- Team members with operations and system administration skills collaborate with other team members to build a CI and deployment infrastructure that supports short cycle times.
- Development teams, including testers, can help implement a test-guided approach to building that infrastructure and ensure that the infrastructure continues to operate effectively.

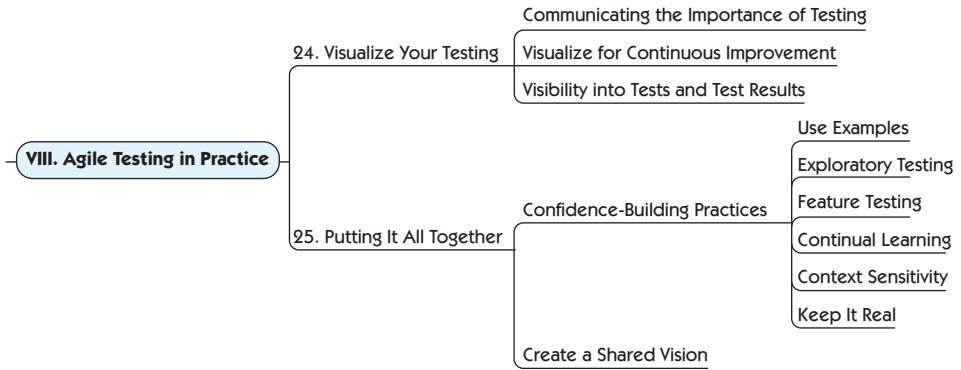
- DevOps practitioners help build and implement test automation drivers, libraries and frameworks, and test data generation tools.
- Understand your team's build pipeline and test environments to take advantage of where you can use automation to supplement your exploratory test efforts.
- Each organization needs to experiment with different hardware configurations to find ones that provide flexibility, consistency, and maintainability.
- Testers can contribute to infrastructure testing by asking good questions and helping with a risk-based approach as well as by actively engaging in testing configuration and deployment.
- Automating the provisioning of configuration base states lets teams do more with fewer resources, using sustainable automation strategies that provide useful metrics and enable more exploratory testing time.
- DevOps can help the team find ways to overcome impediments such as fragile or slow automated tests.

This page intentionally left blank

AGILE TESTING IN PRACTICE

We wrap up *More Agile Testing* with recommendations we feel will do the most to help you and your team build quality into your software products. One crucial practice is making testing activities transparent and providing information that's accessible and easy to understand. Visibility makes impediments immediately obvious, promotes discussion and collaboration, and provides guideposts along the way, as we strive to continually improve our process and our software. In this part we and our contributors share ways to make testing activities highly visible.

Finally, we share practices that build confidence in delivering software products that give value to our customers and our business. Using examples to guide development, exploratory testing, feature testing, continual learning, context sensitivity, and keeping it real while testing help teams gain courage and continually improve. The Pillars of Testing, explained by David Evans, give us the base from which to grow our confidence in releasing software changes.



- Chapter 24, “Visualize Your Testing”
- Chapter 25, “Putting It All Together”

VISUALIZE YOUR TESTING

	Communicating the Importance of Testing
24. Visualize Your Testing	Visualize for Continuous Improvement
	Visibility into Tests and Test Results

One reason agile development works so well is that it makes the development process transparent. Walk into any agile team’s work area, and you’ll likely see some big visible charts. You can tell at a glance what the team is working on, what impediments may be in its way, how much work has been completed, and whether or not the regression tests are currently passing. We devoted a section of *Agile Testing* (pp. 354–66) to preparing for visibility. Here, we’ll explore more ways to increase the visibility of testing in an agile team.

COMMUNICATING THE IMPORTANCE OF TESTING

Agile teams represent work that is planned, in progress, and completed in visual ways, with sticky notes or cards in rows and columns on a wall, or an online project-tracking tool that simulates a physical board. Anyone who looks at the board should be able to understand at a glance what testing activities are planned, which are in progress, and what stories have been tested and accepted.

Making testing tasks highly visible encourages team members to take on uncompleted tasks, even those that fall outside their area of specialty. Impediments can also be clearly represented, so that they are addressed in a timely manner. These boards should be designed to help the team focus on completing one story at a time, or at least a small set of stories. They help teams remember to complete testing activities for a story before rushing to start the next story.

Kanban and Testing

Steve Rogalsky, an agilist at Protegra, explains how kanban boards provide one way to make testing activities highly visible.

Some Background

I'll start by saying that my first agile experiences were Scrum-like. Once I started to understand agile, I quickly gravitated to flow-based systems like kanban. It took me a while to get comfortable with kanban's work-in-progress (WIP) limits. For the first several projects, we ignored WIP limits and instead paid attention to the board—our goal was balance instead of limits. Once we got tired of trying to keep the board in balance ourselves, we switched to WIP limits.

Note that we pair our kanban flow-based approach with a few events that happen every two weeks—demos, planning/design meetings, and retrospectives. Also, we don't currently have trained testers on our team. Our business analysts and programmers share the testing load. We do have customers who do a final acceptance test, but they are trained as business experts and only recently have started testing due to this project. They are learning fast and help us a lot!

Our Board

Our initial boards were pretty basic. "Ready," "WIP," and "Done" columns were soon replaced by "Ready," "Analysis," "Development," "Testing," and "Done" columns. After getting exposure to the "Get Kanban" game, we added a "Ready for Next Status" column.

Our board has gone through a few iterations since then, but Figure 24-1 shows what we are currently using.

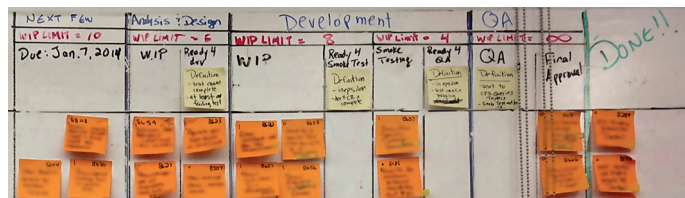


Figure 24-1 Steve's kanban board

The “Analysis & Design” column includes sub-columns of “WIP” and “Ready for Dev.” The goal of this “WIP” column is to write test cases (examples) with our customers to make sure we understand what they expect from this story. The definition of “Ready for Dev” is test cases (examples) written down and reviewed by the customer with at least one failing test. (Of course, at the start, all of them are likely to be failing tests.)

The “Development” column includes sub-columns of “WIP,” “Ready for Smoke Test,” “Smoke Test,” and “Ready for QA.” “Ready for Smoke Test” indicates that the story is deployed to our test environment, it has been code reviewed, and the developer thinks that all tests pass. We define smoke test to mean that an analyst, programmer, or tester has tested the feature and believes that all the test cases pass. A story qualifies for “Ready for QA” when all its smoke tests pass.

The sub-columns of the “QA” column are “WIP” and “Ready for Final Approval.” Here’s where our customer makes a final pass. At this point we would be very surprised if they find a defect or missed test case. If they do find something, we try to fix it immediately and add any missing tests if necessary. The definition of “Ready for Final Approval” is that the customer believes all test cases pass, and the story is complete. The final approval rests with our main customer. She’s pretty busy so she doesn’t test everything, but she wants to have a final look before moving any story to the “Complete” column.

How do we define “complete”? The customer is happy, and it’s ready to deploy!

How It Relates to Testing and Testers

We’ve seen several benefits from using a flow-based system. It provides powerful visualization. Everyone, including testers, can see what is coming up and prepare for it. Since our definitions in most of the kanban columns include the phrases “passing tests” or “failing tests,” it communicates the importance of testing to the whole team and supports a whole-team approach to quality.

The “Ready for” columns offer a lot of freedom from task assignment. We all pull our own work rather than having work assigned by a manager. With almost no exceptions, once we move a card to “Complete,” it doesn’t come back. When the board becomes unbalanced, it supports building cross-functional skills. If testers are low on work, they help programmers where they can, or more often they help analysts facilitate and gather the test cases and examples. When programmers

or analysts are low on work, they generally look to help with testing first. This creates more understanding on the team about the role and importance of testing. Their experience as testers has nudged both programmers and analysts to pay greater attention to the test cases.

Although we aren't doing any significant pairing yet, we keep seeing small instances of pairing—including development-customer pairing during QA. We hope that trend continues.

On the whole, we love this approach. It gives us a lot of pride and confidence. We get the feeling that Deming smiles down on us every time a story moves to “Complete.”

Steve notes that the visualization that the team's board provides is a key benefit. However, Mike Talks reminded us about a potential downside of kanban boards. They allow you flexibility, but if you add too many states to cover all eventualities, you end up draining all the simplicity out of what started as an elegant solution. Constantly review your workflow and ask a question Liz Keogh suggests (Keogh, 2014b): “Is this a useful representation of reality?” If it seems as if tasks are leaping over multiple states, it may be a red flag to indicate that those states may not be relevant.



Figure 24-2 A vertical kanban-style board makes use of the only available wall space.

Use your creativity and experiment with different ways to keep testing activities front and center. Lisa worked on a team where the only wall space was on a column in the middle of the room, and two of the five members were remote. The team put a kanban-style board in a vertical format on the column (see Figure 24-2) and posted photos of it each time a card moved in the team wiki. Lisa, who worked remotely, kept her own copy of it on a cupboard door in her home office. She could see at a glance what to work on next.

A simple story board, such as the example in Figure 24-3 with only four columns—“To Do” (or “Ready”), “In Progress,” “To Review,” and “Done”—is an effective way to visualize testing progress. All tasks, including coding and testing tasks, move along the board. When the team puts in WIP limits at the story level, say only two in-progress stories at a time, team members must help others to complete at least one story before they start the next. When color coding is used to differentiate the testing activities, it is easy to see when they are piled up, either

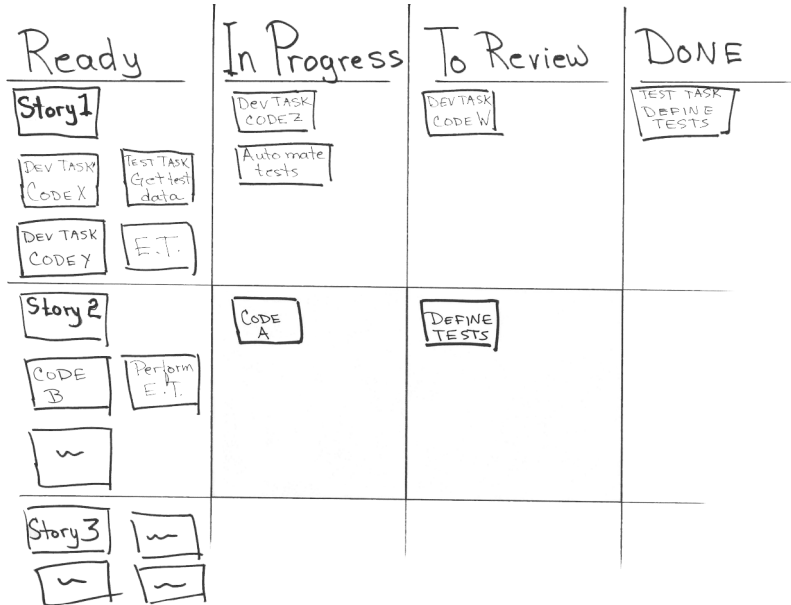


Figure 24-3 Simple story board

“In Progress” or “To Review.” This visibility reminds team members to help out on tasks to get each story finished. Perhaps a programmer can help review tests or pair for exploratory testing.

Dispersed and distributed teams may prefer to use an online board to track testing progress along with other development activities, but we encourage those teams to find a way to keep it in front of everyone as much as possible. Some distributed teams use oversize monitors or projectors to keep their online project-tracking board and continuous integration (CI) dashboard on view in each team area.

We provided more story board examples in Chapter 15 of *Agile Testing*, and an online search of “agile big visible charts” or “agile story boards” will provide more examples to fuel your imagination.

VISUALIZE FOR CONTINUOUS IMPROVEMENT

One of the Ten Principles for Agile Testers we described in *Agile Testing* is to practice continuous improvement. We should always be looking for ways to do a better job. Use big visible charts to show goals for improvement, experiments in progress, and results of completed experiments.

In our experience, if your team has an obstacle in its way, it helps to make it visible. When it’s “in your face,” you remember to do something about it. When you identify areas for improvement in your retrospectives, set SMART goals (specific, measurable, achievable, relevant, time-boxed) or find some way to make the problem more visible. This sets the stage for continuous improvement.

Using Kanban to Continually Improve

Mary Walshe, a tester at Paddy Power in Ireland, shares her team’s experiences with using a kanban system to visualize areas of improvement.

We have been using kanban in our department for well over a year now and have made huge improvements in our process. We use it to visualize areas that we could improve, to tune our process, and to see where we can experiment. It’s a tool that the whole team owns and

that we use for continuous improvement. If we notice that the board has not changed in a while, we know that we may be slacking on our experiments.

One of our most recent improvement goals was to try to reduce our cycle times by three days. Our kanban board, and its WIP limits, helped us identify an area of possible improvement. We saw there was a bottleneck in our “Awaiting UAT” column, and conversations began around the duplication of effort in this area.

Multiple user stories delivering business value were spending over three days in acceptance. All that business value could have been getting to our customer three days earlier!

The acceptance bottleneck impacted product owners, developers, business analysts, operations staff, testers—in short, everyone! Product owners spent more time preparing, executing tests, and documenting the activity. Customers had to wait longer for business value to be delivered. Long cycle times meant a longer feedback cycle. Development slowed down. Repeating tests already done during exploratory testing slowed things down.

Our product owners suggested that we try to “pimp the demo,” in other words, focus on satisfying product owners with the demo of newly completed work. We did some research and decided to try introducing Demo 2.0. A hypothesis story for this might read something like what is shown in Figure 24-4.

PADDYPOWER.	
We Believe That	Demo 2.0
Will Result In	Shorter cycle times
We Will Know We Have Succeeded	
When	Cycle times from 'Demo' to 'OAT' have decreased

Figure 24-4 Shorter cycle time hypothesis

A few members of the team took it upon themselves to work with our product owners and ensure that Demo 2.0 would satisfy their needs. They wrote a demo script and sent it along with the acceptance test report in the invitation to the demo, which was scheduled in advance with the product owner. During the demo, we walked through the acceptance tests for each feature and executed the demo script. We allowed time for product owners to ask questions and do extra tests as needed.

So that's it. Seems pretty simple, right? As of this writing, it's been nearly a month since the first Demo 2.0. Looking back at our hypothesis story, we have indeed succeeded by decreasing cycle times from demo to operational acceptance testing (OAT).

Let's look at the metrics. In Figure 24-5, we can see that the average cycle time (the lower line) from demo to OAT in September was three days.

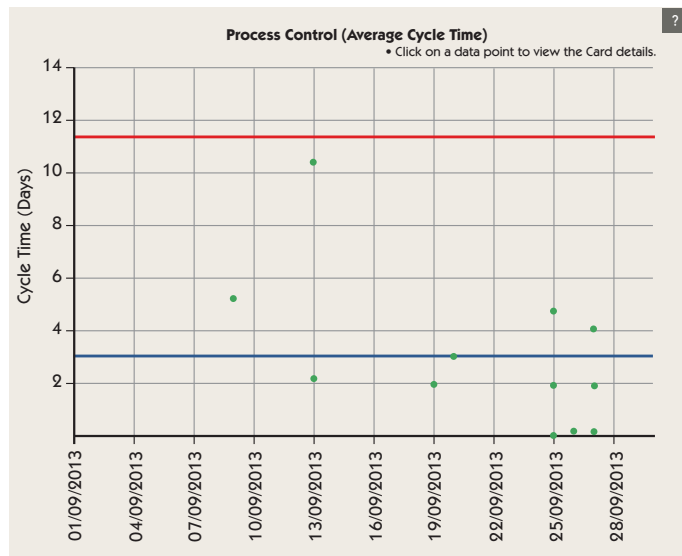


Figure 24-5 Average cycle times from demo to OAT before Demo 2.0

Figure 24-6 shows the cycle times from demo to OAT starting on October 1, the day of the first Demo 2.0. We can see that the average cycle

time from demo to OAT in this month has come down to under one day. (Note: The scale on the y axis was adjusted to provide a better visualization of the differences.)

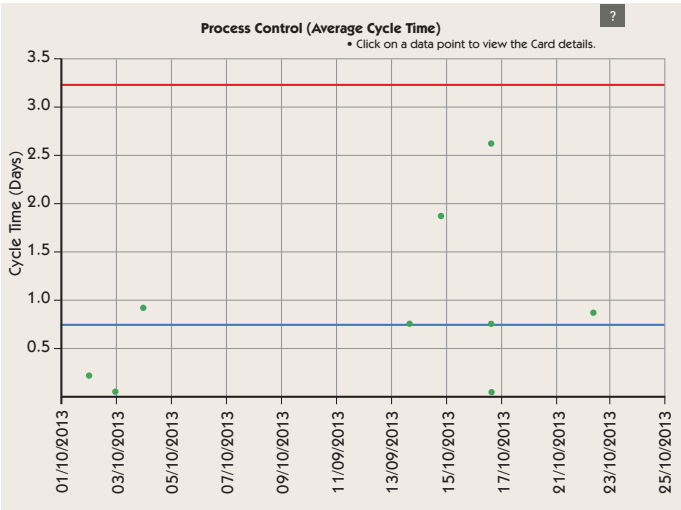


Figure 24-6 Average cycle times from demo to OAT after Demo 2.0

We can see the decreased time via metrics from taking the cycle times from demo to OAT from the previous month of September and comparing them to the current month of October where we introduced Demo 2.0. So, have we succeeded? Yes, we knocked two entire days off our cycle time with this one initiative!

Critics might wonder if the longer demo added to the cycle times from starting a story to accepting it, counteracting the saving of two days, but the metrics show that the demo hasn't increased cycle times. In fact, the time taken in the demo lane has decreased in the last month. This may be due to the fact that reducing waste is in the forefront of the team's mind.

The whole team put a lot of effort into getting this off the ground, including the initial organization of what the new demo would look like, putting it into action, getting feedback, and improving it along the way. All in all, I think Demo 2.0 has been a great success. In the future, we might look at whether we even need the acceptance phase.

Steve's and Mary's stories have much in common. The message isn't that everyone should use a kanban system. Rather, it's essential that a team be able to visualize its testing work and quality level and make this information transparent for the business. You can try any type of story or task board or online tracking system to see if that gives you the visibility you need. It might involve a big monitor displaying live CI results in your team room or something as simple as a whiteboard drawing, spreadsheet, checklist, or wiki page. The main criterion is making it front and center for the whole team and all stakeholders. This transparency helps teams easily identify the biggest problem to work on next. They can brainstorm experiments to chip away at that problem and continually improve their testing efforts.

Don't wait until the end of the iteration to start demonstrating small increments of completed work to your stakeholders. Frequent collaboration helps you stay on track and avoid wasting time.

VISIBILITY INTO TESTS AND TEST RESULTS

We've often heard development teams complain, "We don't know what the testers are doing!" To us, this translates into, "We have no confidence in what is being tested." There are many ways to show what is being tested.

We talked about using testing mind maps to brainstorm with the team in Chapter 7, "Levels of Precision for Planning." Figure 24-7 shows a mind map Lisa and another tester created on a rolling whiteboard to visualize a new batch account fee collection process, which would be integrated into an existing transaction processing system. As they created the mind map, they thought of questions to ask the product owner and programmer. The team used the mind map over several iterations as they worked on the new features. Whenever questions came up, team members would gather in front of the mind map to discuss them. As testing was completed for each mind map node, they checked it off on the whiteboard, making progress visible.

Teams can expand initial acceptance tests to give the whole team visibility into testing. Making them accessible to everyone fosters

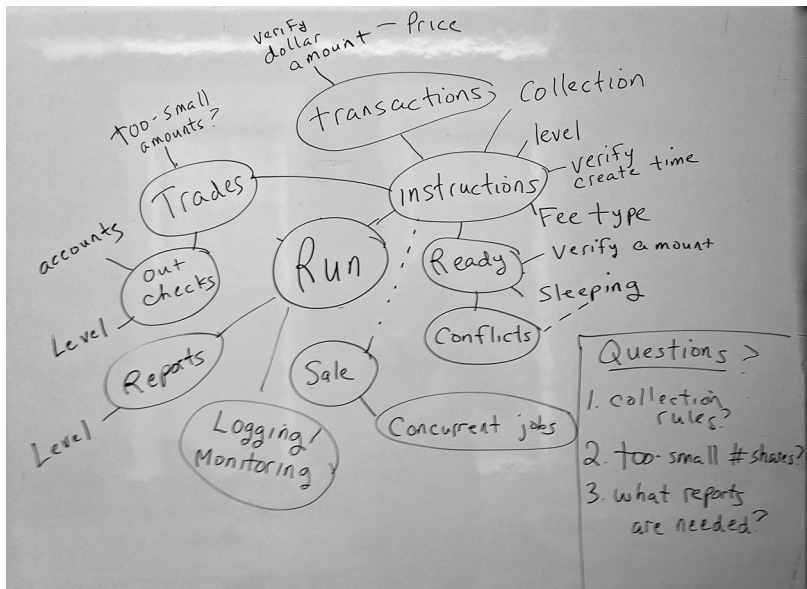


Figure 24-7 A testing mind map

collaboration and encourages developer and customer team members to comment and suggest new ideas.

In Chapter 7, “Levels of Precision for Planning,” Figure 7-6 showed an example of a test matrix. This is a simple way to make high-level testing visible to stakeholders outside the team. It also gives a different perspective on testing to help stay away from the problem of getting caught up in stories and forgetting the big picture.

Testing is about providing information as quickly as possible. Think about how you can make the information learned from testing visible. How can you communicate it? How will people in other roles hear and see it? As we discussed in *Agile Testing* (pp. 357–66), visible test results are one of the most important ways we can measure progress.

This information takes many forms. Continuous integration provides a critical snapshot of the current status of your builds and perhaps your automated test runs. Results from exploratory testing sessions can be

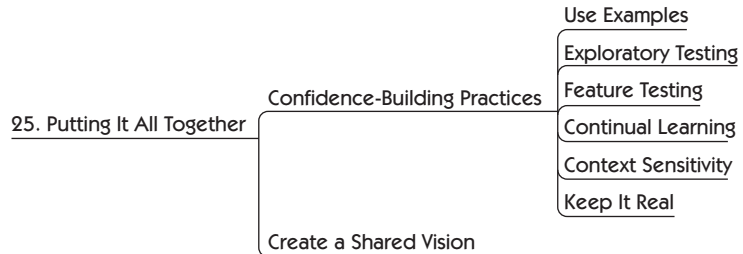
consolidated in a shared document, a team wiki, or a simple test dashboard. Tests and results can be noted on the story or featured in an online tracking tool. Bug fixes can be captured as automated tests, and defects not fixed immediately can be tracked in as simple and visible a way as possible. Most importantly, test results should be discussed at team standups. Use the feedback to guide your next steps.

SUMMARY

The transparency that agile provides is one of its biggest benefits to an organization. Testing is about providing information, and when we make that information easy to see, everyone can talk about it. It's not enough "just" to be visible—it needs to be useful.

- Look at ways to make various aspects of testing more visible on your story, task, or kanban-style board. If your process has a definition of "ready" and a story isn't on the board until it meets that definition, it's easy to see when a new story is ready for the testing tasks to be addressed.
- Experiment with different styles of physical story boards and, if necessary, online tracking tools to see what provides the most visibility into testing progress for your team.
- Demonstrate new work to customers early and often to make progress visible, get quick feedback, and reduce story cycle times.
- If your team encounters obstacles to successfully completing testing activities, find a way to make those obstacles visible. It can be as simple as a bright red blocker card on a story board.
- Consider techniques such as mind maps that help teams visualize their work and progress. Visual representation helps teams stay grounded in reality.
- Testing generates information about quality and risk. Use a variety of ways to make test results and current status visible to everyone.

PUTTING IT ALL TOGETHER



Agile Testing wrapped up with seven key success factors for testing in agile projects. In fact, we feel they are critical for success in any project, whether you call it agile or not. They are:

1. Use the whole-team approach to quality.
2. Adopt an agile testing mindset.
3. Automate regression tests.
4. Provide and obtain feedback.
5. Build a foundation of core agile practices.
6. Collaborate with customers.
7. Look at the big picture.

We and our contributors have shared in this book what we've learned since *Agile Testing* was written, addressing challenges that are becoming more common for teams. The original success factors apply to large enterprise environments, globally distributed teams, testing in different contexts such as mobile and embedded software, data warehouses, and continuously deployed products. However, we have more and better tools and techniques available to help with ongoing challenges.

CONFIDENCE-BUILDING PRACTICES

While we don't believe there are any "best practices," we do believe there are core testing practices that benefit most agile teams. These practices give teams confidence to deliver quality products that delight their customers. With that confidence comes the courage to try new things and continually improve.

Use Examples

Learn to guide development with examples, using acceptance-test-driven development (ATDD), specification by example (SBE) or behavior-driven development (BDD). Make your examples as concrete as possible using visual means where possible (see Figure 25-1).

Among the core development practices we recommend are test-driven development (TDD) and continuous integration (CI). These go a long way toward building quality into the software and learning from fast feedback. However, by themselves they are not enough. When delivering a new feature is complicated, guiding development with business-facing

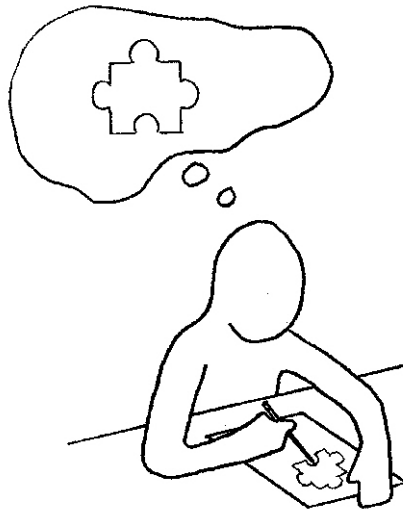


Figure 25-1 Use real examples.

examples helps prevent stories from being rejected multiple times. When proposed features have high uncertainty or are so complex that it's hard to even come up with examples of desired behavior, you may need some short cycles of spikes and learning before you can distill the information into concrete examples.

We can help our business experts identify the most valuable software features to implement by combining testing and business analysis (BA) mindsets. Agile techniques such as story mapping and impact mapping help the business and development teams focus the team's effort on the right goals. BA techniques such as the 7 Product Dimensions described in Chapter 9, "Are We Building the Right Thing?," further our efforts to create valuable products. Although we don't want to waste time with big up-front analysis of ever-evolving requirements, there are good, quick ways we can flesh out story details before we start coding a particular feature or story.

This information can be turned into tests that guide development, many of which can be automated and form part of the regression suites. It helps ensure that we deliver useful value and helps us build in short feedback loops to learn more as we develop and deliver incrementally. We still have more testing to do once we think coding is finished, but we've minimized the chances of rejection without a huge up-front cost. Part IV, "Testing Business Value," described techniques to help extract desired behaviors and misbehaviors from customers and learn to guide development with examples.

If CI is the heartbeat of an agile team, guiding development with examples is the heart monitor that shows we are on the right track.

Exploratory Testing

Although some early agile teams had testers who did some exploratory testing, the value exploratory testers contribute through their ability to see beyond the story functionality has become more visible in recent years. Testing for business value (testing early) is about preventing defects—exploring ideas if you like. We, however, refer to exploratory testing as testing the software—if you can't prevent defects, find them as early as you can (see Figure 25-2).

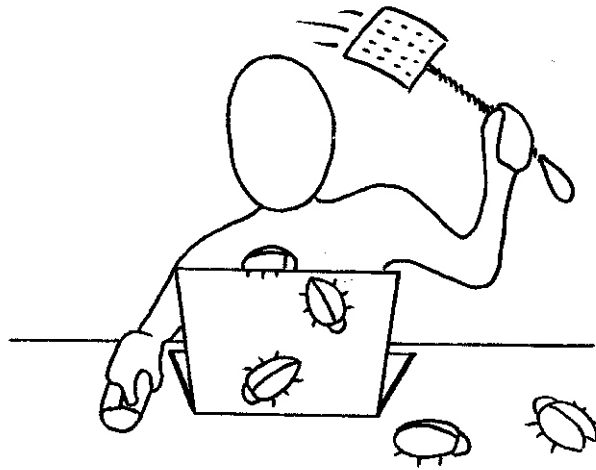


Figure 25-2 Explore.

Chapter 12, “Exploratory Testing,” discussed several exploratory testing techniques. We encourage you to use the links provided in the bibliography for Part V to experiment to discover what works best for your situation. If some members of your team aren’t familiar with exploratory testing, hold a workshop or testing dojo to let them learn and practice techniques.

Some exploratory techniques are ideal for collaborating with other disciplines on the team. Involve your user experience (UX) designers to create user personas. Get testers, programmers, and analysts together to create and prioritize testing charters. Share information learned in exploratory sessions with the developer and customer teams, making the information visible and actionable. Experiment with different approaches, and see what adds the most value for your team.

Feature Testing

Incremental and iterative development is a cornerstone of agile. Slicing stories into a tiny, consistent size helps teams make predictable progress, but in these small components of stories, we may lose the bigger picture. Looking at the big picture is a key success factor. We need techniques

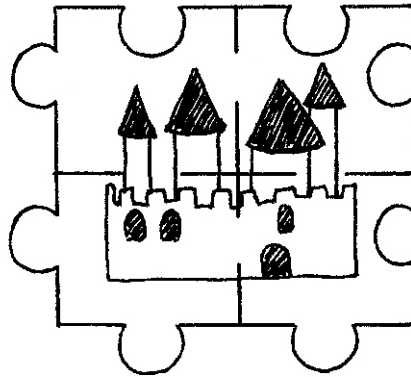


Figure 25-3 Remember the big picture.

to test at the feature level once the building blocks are complete (see Figure 25-3).

Janet often compares feature testing to assembling a jigsaw puzzle. You don't necessarily have to piece together the entire puzzle to derive value from it. She focuses on the parts of the picture that are the most meaningful to her: faces of people in the picture, beautiful sections of scenery (Gregory, 2011).

When we test software features, we must identify the areas of most value to the end users, the parts that “have to work,” and focus our testing there. These may not be in the actual functionality; performance, security, or convenience may be what the customer values most.

As your team discusses your next big feature set, think about the different types of testing explained in Chapter 13, “Other Types of Testing,” as well as different models such as the agile testing quadrants. Write stories with testing activities that focus on aspects of the feature that users value most. Consider the quality attributes and constraints that affect testing during team conversations.

Continual Learning

Taking time to experiment, using short feedback cycles to evaluate what works and what doesn't, having a sense of personal safety that failure

will not be punished—these are all critical components of an environment that nurtures learning (see Figure 25-4). As a team survives its initial transition to agile development and continues over the years, new problems and challenges will arise. To succeed over the long haul, we need to keep improving. We can't do that if we don't learn.

This applies to all disciplines on an agile team, of course. But in our experience, testers and testing are often forgotten in the rush to implement agile development. Testers often don't get any special training. They may also get trapped in a mini-waterfall situation, where stories aren't delivered to test until the last day of the iteration, so that testing is always playing catch-up.

When we're focused on delivering new features, it's hard to step back and take time for learning. Yet if we don't devote time to learning, we may not be able to keep our customers happy.

If you're in a management or lead position, you have a chance to nurture a learning culture. Regardless of your role, try to get your team to discuss areas of testing where the team lacks expertise, and think of ways each team member can grow his or her "T-shaped skills" (as described in Chapter 3, "Roles and Competencies") to cover the missing capabilities.



Figure 25-4 Learning

Context Sensitivity

Be aware that you may need to apply agile principles differently depending on your context. This is one of the main areas we explored in this book, although we're sure we didn't touch on all the different situations out there; new challenges come up every day. Understand your context (see Figure 25-5), and strive to apply agile principles and practices to your environment. (*Note:* We use the word *context* per its *Merriam-Webster* definition as “the situation in which something happens: the group of conditions that exist where and when something happens. We're not discussing “context-driven testing” (Kaner, 2012), though you may want to learn about that, too.)

For example, if you work in a large corporation, you may have to adapt testing around existing organizational controls. You may need additional roles to help coordinate testing activities and manage dependencies across the organization, or maybe even have additional teams for activities such as system-wide testing encompassing multiple applications. More support from the executive level can help effect cultural changes to switch testing mindsets from bug detection to bug prevention.

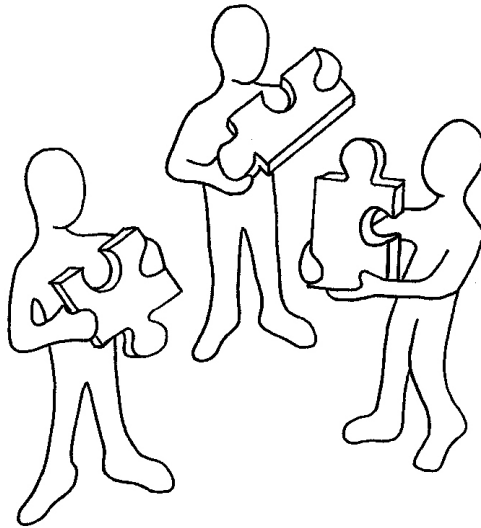


Figure 25-5 Know your context.

Some software teams must satisfy regulatory requirements for specific documentation. If teams think about the simplest way to provide the necessary evidence required by auditors, they may be able to reduce both the amount of documentation and the time spent on it to a minimum. It may seem counterintuitive at first, but agile testing values and practices work well in this context to comply with regulations and reduce risk for the product.

Distributed teams require innovative approaches to enable adequate collaboration and communication in their context. It's hard for a distributed team to bond if team members never have a chance to meet in person or talk directly to each other. If your team is geographically dispersed, look for creative ways to build trust and enable collaboration so that you can benefit from a whole-team approach to testing. Using tests to help define the specifications is one way to get a better shared understanding of what to build.

Mobile and embedded software teams face extra testing challenges because of the number of platforms and devices supported and many other variables, such as how and where the devices will be used. These teams can improve software quality through collaborating across roles, leveraging automation, and using fast feedback loops.

Business intelligence and data warehouse teams are tasked with making sure business experts have high-quality data for planning and making business decisions. It can be difficult to figure out how to build these systems incrementally and iteratively. Specialized technical skills and business domain expertise may be needed to succeed with agile testing in this context.

New specialized skill sets that focus on testing and quality from an operational point of view have been brought into delivery teams. The whole team engages in DevOps activities to streamline delivery, shorten feedback cycles, improve automated test effectiveness, and ensure that CI, test, and deployment infrastructures support short cycle times. Our software product is more than just the application under test, and DevOps helps expand our testing to encompass all facets of the product and the value it provides to customers. Whatever your team's context, DevOps can help you continually improve software quality.

Keep It Real

At some point we all aspire to have superpowers, and our managers often seem to think we possess them! But as Bob Martin reminds us in *Clean Coder* (Martin, 2011), the best way to be a team player is to inform management what we can realistically accomplish and help them prioritize the highest-value work.

Even if your team doesn't use a flow-based process, the concept of limiting work in progress (WIP) is essential. Each of us has only so much bandwidth. If the testers on the team can test only a maximum of two user stories at any time, it doesn't make sense for the programmers to keep cranking out stories. If the programmers stop new development and help with testing, the team can move forward much more easily. Make testing part of each story and each feature.

Don't succumb to the desire to make customers happy by saying, "We'll try" when you know it's impossible. It takes courage to say no! Experiment with ways to make the testing WIP limit visible. Rather than shrug and say, "We have a testing bottleneck," figure out ways to remove the constraint, such as having the whole team help with testing activities to keep work items moving through at a steady pace. Help your team take a reality check when needed (see Figure 25-6).



Figure 25-6 Reality check

CREATE A SHARED VISION

Your own team, its environment, its constraints, and its shared vision are unique. There's no magical set of agile testing practices that guarantees success. However, agile values and principles guide us. Trying small experiments and building in short feedback loops help us stay on track and make course corrections along the way.

We've described some core testing practices that, in our experience, are valuable for most teams. We find it helpful to take a step back and remember the purpose of testing.

Pillars of Testing

David Evans, *an agile quality coach from the UK, reminds us about the purpose of testing and sums it up beautifully with his "Pillars of Testing."*

Testing does not produce quality; it produces information about quality. At a high level, the outcome we want from testing is confidence in our decisions to release software changes. Better testing produces greater confidence in those decisions. Therefore, the valuable product of effective testing is Confidence.

In my "Pillars of Testing" model (see Figure 25-7), I map out the connections and dependencies of a number of factors that are critical to raising this confidence. I represent the model as a "temple," with a ceiling, four pillars, foundations, and bedrock.

At the top of our model, Confidence rests upon Courage and Safety. Courage without Safety leads to foolhardiness; Safety without Courage leads to stasis and inaction.

The four pillars that push up our level of confidence in testing are

- **Stronger Test Evidence:** How well do our stakeholders understand what has been tested and when?
- **Better Test Design:** How appropriate, meaningful, and effective are the test cases we use?
- **Higher Test Coverage:** How much of the system is tested compared to our models of risk?

- **Faster Test Feedback:** How quickly do we identify problems and their remedial actions?

Each pillar represents qualities that exist on a scale—they all exist to some extent, but they can always be improved and raised higher. The more of these qualities we have, the more Confidence we create. We must also be conscious of balance and avoid overinvesting in any one pillar to the detriment of others.

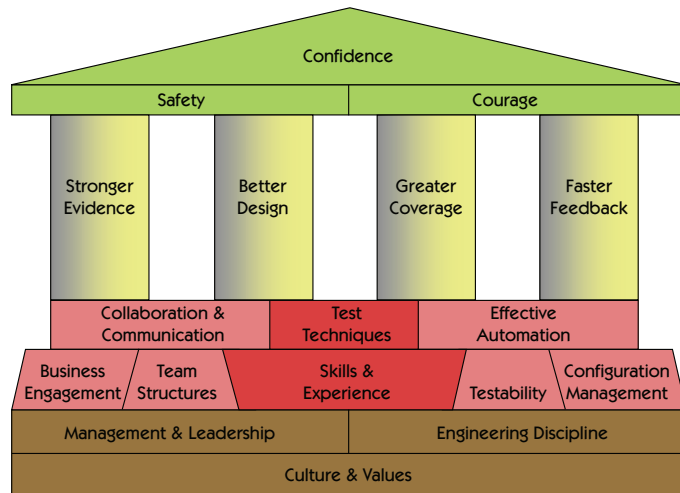


Figure 25-7 Pillars of Testing

Supporting the Pillars are the Foundations, representing three key areas: Team Foundations, Capability Foundations, and Technical Foundations. Each area includes several interconnected building blocks:

- **Team Foundations**
 - **Collaboration and Communication:** This represents the extent to which a team is able to get the most out of a variety of perspectives and skills, both within and beyond the team, and to find the most effective and informative communication modes among all stakeholders.